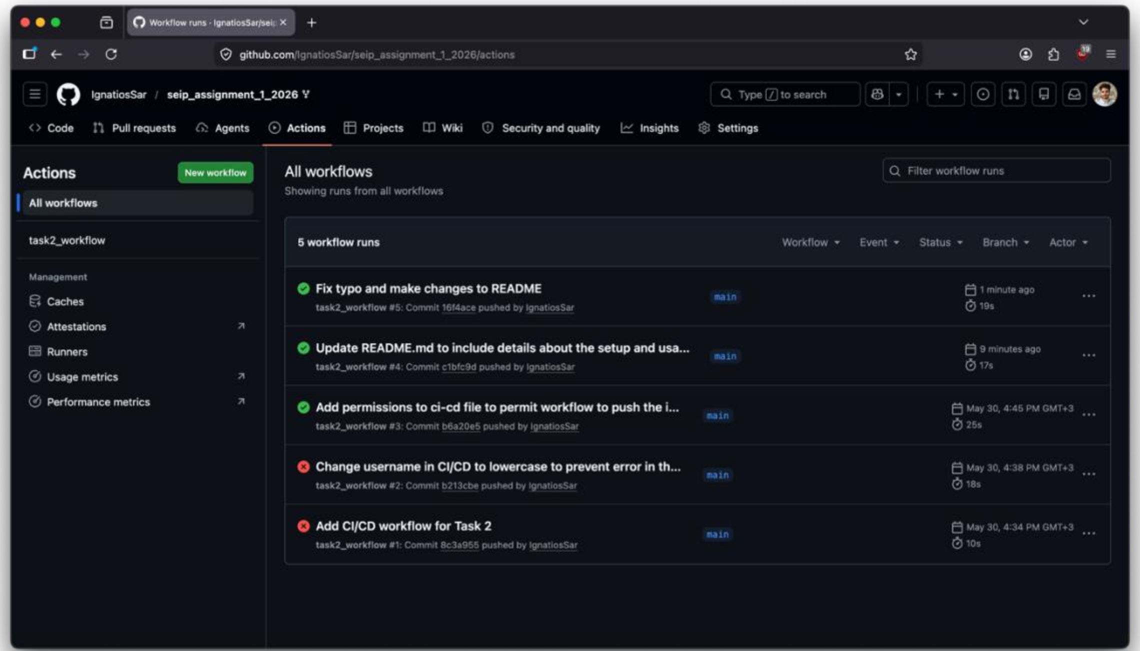
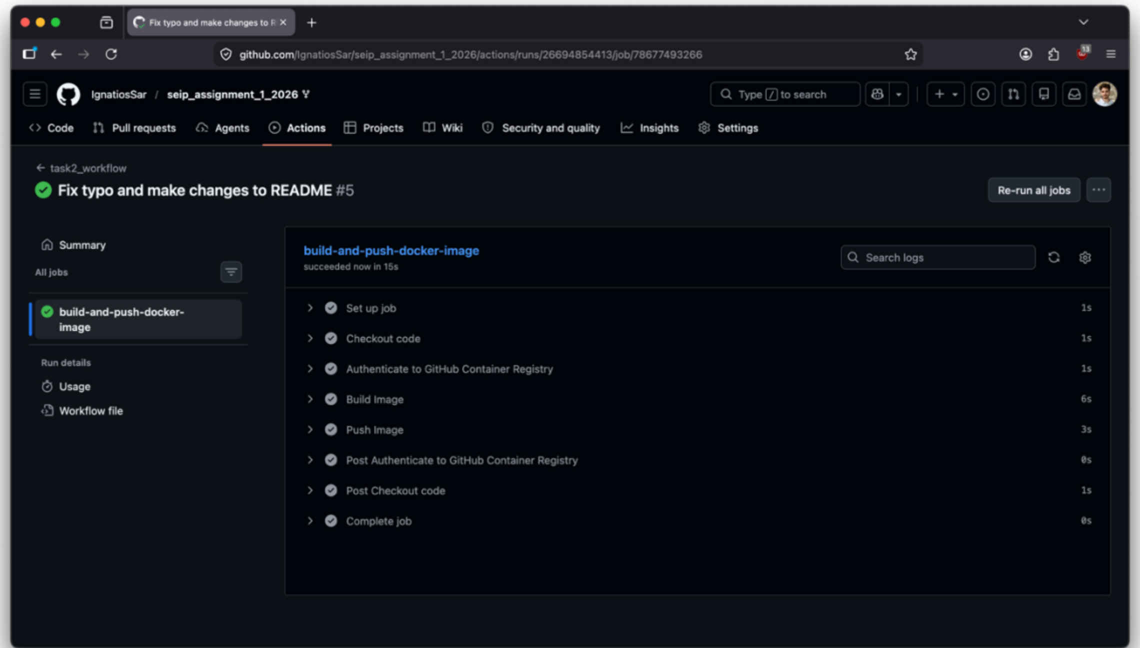
SEIP ASSIGNMENT 1

IGNATIOS STYLIANOS SARANTIDIS

8230131

1. **GitHub Repository Link:**
   https://github.com/IgnatiosSar/seip_assignment_1_2026
2. **CI/CD Proof**





3. Cluster State Proof

```
ignatios@192:~/Projects/seip_assignment_1_2026
➜  seip_assignment_1_2026 git:(main) ✗ kubectl get all -n default
NAME                                          READY   STATUS    RESTARTS   AGE
pod/echo-api-deployment-584c856ff7-9hwzd      1/1     Running   0          91s
pod/echo-api-deployment-584c856ff7-x62ps      1/1     Running   0          91s
pod/echo-api-deployment-584c856ff7-xgwwg      1/1     Running   0          91s

NAME                       TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
service/echo-api-service   ClusterIP   10.99.153.190   <none>        80/TCP    91s
service/kubernetes         ClusterIP   10.96.0.1       <none>        443/TCP   4m24s

NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/echo-api-deployment   3/3     3            3           91s

NAME                                            DESIRED   CURRENT   READY   AGE
replicaset.apps/echo-api-deployment-584c856ff7   3         3         3       91s
➜  seip_assignment_1_2026 git:(main) ✗ kubectl get configmap,secret
NAME                          DATA   AGE
configmap/echo-api-config     2      111s
configmap/kube-root-ca.crt    1      4m38s

NAME                     TYPE     DATA   AGE
secret/echo-api-secret   Opaque   1      111s
➜  seip_assignment_1_2026 git:(main) ✗
```

4. Application Verification Proof

```
ignatios@192:~
kubectl (kubectl)   ⌘1        ~ (-zsh)   ⌘2
Last login: Wed May 27 13:25:10 on ttys005
➜  ~ curl http://localhost:8080/
{"message":"Welcome to the Software Engineering in Practice Assignment Cluster!","environment":"production"}%

➜  ~ curl http://localhost:8080/secure-config
{"status":"Authorized","injected_secret_suffix":"**********2026"}%
➜  ~
```

5. **AI Usage & Future Engineering Report**
   - **AI Integration & Utility Analysis**
     I used GenAI and in particular Gemini, to assist me and guide me through the tasks of the assignment. I asked it to help me understand what Minikube & kubectl are and how to use them. It also helped me understand certain concepts such as manifests, what Base64 encoding is and how it is used in this context, what checkout is in CI/CD and what it is used for and to explain in detail examples that I found for Dockerfiles, workflows and manifests. I also used it as reviewer before my commits to ensure that my code had no mistakes and it meets the requirements of each task. Finally, it helped me with debugging, especially in Task 2, where I got a permission error when running the workflow.
   - **Friction Points**
     Although for the most part Gemini's responses were accurate, there were 2 friction points that I encountered. Firstly, when it gave me examples It used outdated versions e.g in ci-cd workflow, in an example about checkout, it had v4 instead of v6. To get past this, I searched for both documentation and examples to find the latest versions. Secondly, it failed to find me 2 defects when I asked to review the code in the ci-cd workflow, resulting in failure when running the actions. I resolved this by looking at the logs, finding the faulty code and making the required changes.
   - **Future Architectural Outlook**
     If I were to scale this pipeline for a real world system, I would do the following.
     - Integrate an external secret manager like HashiCorp Vault or AWS Secrets Manager, to truly encrypt sensitive data saved in secret.yaml.
     - Replace port-forwarding with an Ingress Controller like NGINX Ingress, to be responsible for load-balancing and request routing, without requiring a constantly active terminal session.
     - Establish a GitOps Workflow with a tool like ArgoCD to automate the application of manifests and the synchronization of the cluster with the main branch of the repository.
     - Add a monitoring tool like Prometheus to constantly check the health and status of the pods in the cluster.
     - Integrate a security scanner like Trivy, to scan for CVEs and prevent exposure to supply chain attacks.
     - Introduce an autoscaler like HPA, to dynamically scale the necessary pods.